Aegis OCI provisioning and deployment

Final deployment handover for Swamy
Windows PowerShell | Branch ags_sprint1 | 16 October 2026

Use this guide to prepare a new OCI destination and release the selected Oracle database objects, frontend and backend code. The separate provisioning section explains the current pilot example and first-install handoff. For code/object releases, choose the AI skill or JSON plus PowerShell; both preserve existing infrastructure, application settings, stored data and model registrations.

Two supported ways to deploy

Route	What Swamy does	Page
AI skill	Give the target inputs to aegis-deploy; approve discovery and then the complete release plan.	6
JSON + PowerShell	Fill one target JSON file and run deploy.ps1 in an interactive terminal.	7

One terminal command from the repository root:

```
.\ags_deploy\deploy.ps1 -Inputs C:\aegis-target\target.json
```

Scope: DB_Object Y creates only listed missing objects, and a missing application user/schema when the release file permits it. DeployFrontend Y updates web. DeployBackend Y updates API and worker together. Each component can be selected separately.

Prepared OCI is required. The physical Autonomous AI Database, OKE workloads, storage, Secrets, saved connections, model assets and routing must already be prepared. This command cannot turn an empty tenancy or bare cluster into a working installation. Missing initial setup must be completed separately by the platform/application owner.

Existing configuration is preserved. No infrastructure provisioning, runtime configuration changes, model downloads, existing user/grant changes, data import or automatic rollback are part of this release. No backup is created or required by this workflow. Actual image updates can briefly interrupt service with the existing Recreate strategy.

Choose your starting point

New OCI destination: prerequisites and purpose tables on pages 9-10; separate one-time provisioning and initial setup on pages 11-14. Then use the release workflow.
Prepared destination: pages 2-5 cover tools, inputs, flags and database behavior; page 6 is the AI route, page 7 the terminal route, and page 8 verification/recovery.
The AI skill and deploy.ps1 still exclude infrastructure provisioning. Adding this reference section does not authorize infrastructure or configuration changes.

1 Confirm the destination is prepared

For another tenancy or database, obtain that destination's access and configuration. Do not reuse the original live wallet, owner, kubeconfig or endpoint. Tenant details alone are insufficient. Ask the platform/application owner to confirm the items below before publication.

Prerequisite	Required handoff
Oracle database	An existing Autonomous AI Database supporting the maintained native VECTOR, SQL BOOLEAN and collation definitions; reachable with the target wallet and authorized account.
Application workloads	Existing Aegis Deployments and container names, namespace, Services, persistent claims and shared API/worker state. Ready nodes alone are insufficient.
Saved configuration	Target DB connection and wallet, Object Storage route, encryption key, credentials and runtime settings already installed. Existing model registrations and CPU reranker assets must be available.
Access and registry	Destination kubeconfig/context and its authentication executable; authorized Kubernetes access; existing registry repositories, push login and cluster pull access.
Public access	Prepared HTTPS routing and hostname. The tool can discover an existing URL; it does not create DNS, certificates, Ingress or a load balancer.

For a missing application schema, the release plan may label app readiness as conditional. After approved DDL, the saved connection and application must pass checks **before images are published**. Creating tables alone does not seed application users, models, documents or configuration.

The full OCI resource and model prerequisite tables, with each item's purpose and the observed pilot example, are on pages 9-10.

Prepare the release computer

Use Windows PowerShell 5.1 or newer, Python 3.10 or newer (3.12 recommended), and Git. DB object creation needs the Oracle Python driver. Application releases additionally need Docker in Linux-container mode, kubectl, registry login, and the authentication executable named by the destination kubeconfig. Standard OCI CLI kubeconfigs require OCI CLI and its existing identity.

Keep the wallet, kubeconfig, private keys and target JSON outside Git. The current Mumbai pilot kubeconfig uses a private local authentication helper/state; it is not portable to another tenancy. Obtain a destination-specific kubeconfig from its owner.

Database-only selection: no Docker, Kubernetes, registry or application URL is required. Use DB_Object Y, DeployFrontend N and DeployBackend N. Missing prerequisites stop the release; resolve them outside this code/object-only scope.

2 Prepare source and target inputs

Obtain GitHub access to the repository. On a new release computer, run these commands individually and stop on errors. If you already have a checkout or virtual environment, reuse it without overwriting pending work.

```
git clone --branch ags_sprint1 https://github.com/Maniatramco/Aegis.git
Set-Location .\Aegis
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install `
  -r ags_backend\requirements-oracle.txt
New-Item -ItemType Directory -Path C:\aegis-target -Force
Copy-Item .\ags_deploy\deployment-inputs.example.json `
  C:\aegis-target\target.json
git rev-parse HEAD
```

Fill the copied file with known destination values. Blank supported fields are prompted or discovered after permission. Ref may be an exact tested commit; ags_sprint1 selects that branch's resolved commit. The release builds an isolated Git export of that commit, excluding uncommitted and untracked files.

JSON fields	What to supply
SourcePath / Ref	Absolute local repository path; tested branch, tag or commit. Record the exact commit shown in the plan.
Kubeconfig / Context / Namespace	Destination kubeconfig path, its exact context, and existing Aegis namespace.
Registry / RegistryNamespace	OCIR hostname without https://, and tenancy registry namespace. This is different from the Kubernetes namespace.
Platform / AppUrl	linux/arm64 or linux/amd64 matching nodes. Existing HTTPS URL; leave blank to discover prepared routing.
WalletPath / DatabaseOwner / DBAdminUser	Destination wallet ZIP, application schema and existing inspection/admin account. New-schema creation needs account creation and grant privileges.
ServiceAlias / ExpectedService	Wallet TNS alias and full database SERVICE_NAME from the target descriptor. They must identify the intended database.
ValidationReference	Truthful test/CI evidence for the exact source. The deployment command records this reference; it does not execute the regression suite.

The template contains the three Y/N selectors and DB_ReleaseFile as shown on page 4. Do not add passwords, tokens, private keys or wallet contents to JSON. If using the separately filled Mumbai JSON, remember it targets the **existing live installation**; replace its destination values for another OCI environment.

3 Select exactly what to deploy

Use the following selection fields inside the full target JSON. The remaining destination fields come from the template on page 3. Use quoted Y or N values.

```
{
  "DB_Object": "Y",
  "DB_ReleaseFile": "ags_database/release.json",
  "DeployFrontend": "Y",
  "DeployBackend": "Y"
}
```

Release selection	DB_Object	Frontend	Backend
All three	Y	Y	Y
Database only	Y	N	N
Frontend only	N	Y	N
Backend API + worker only	N	N	Y
Both code components; no DB DDL	N	Y	Y

Backend includes both API and worker. Unselected images are not built, pushed or changed. All three N is rejected. With DB_Object N, a backend release still checks schema compatibility and active jobs read-only; frontend-only with DB_Object N makes no Oracle queries.

The database release file controls the script list

`ags_database/release.json` contains **CreateSchemaIfMissing: "Y"** and an ordered **Scripts** array. The supplied manifest lists 13 maintained SQL files under ags_database/ags_tables, containing 13 table and 20 explicit index definitions. Only these listed files participate; the tool does not scan a folder for more scripts.

A repository-relative manifest and all SQL paths are resolved from the selected committed source. An absolute external manifest is also supported: its content is frozen in the private record, while its SQL paths still refer to that committed source. Edit and commit repository SQL/manifest changes before selecting their release ref.

This release supports the maintained CREATE TABLE/INDEX definitions. Arbitrary migrations, stored-program releases, ALTER/DROP statements and DML scripts require separate implementation and review. Do not add them expecting a general SQL runner.

4 Review database and runtime behavior

When DB_Object is Y

Read-only preflight compares the listed object definitions against the intended owner and full service. Matching tables and indexes are skipped. Missing supported objects are created in manifest order only after approval. Incompatible definitions stop the release before DDL; row counts or table counts alone are not a schema comparison.

If the application user/schema is missing and CreateSchemaIfMissing is Y, the plan shows the new account, CREATE SESSION and CREATE TABLE grants, existing DATA tablespace and default 256 MB quota. Optional SchemaTablespace and SchemaQuotaMB select other reviewed existing-space settings. Existing accounts, passwords, grants and quotas are preserved. No physical database or tablespace is created.

Wallet, admin and application-owner passwords are entered privately when requested in the interactive terminal. A new schema password is confirmed privately. They are not saved in JSON, argv, generated SQL or release records. The database applies its password policy.

No existing application rows are imported or changed by this object release. A missing schema is an empty application schema; it does not migrate the original workspace. Database-only with every listed object already matching finishes as verified-only without PUBLISH because there is no change to apply.

Use the committed live-matching build profile

Area	Maintained release profile
Backend base	Python 3.12.15 with the immutable base digest matched to the observed live ARM64 image.
Frontend base	Node.js 22.23.3 with the immutable live-matching base digest; production npm ci uses package-lock.json.
Backend dependencies	Committed pinned requirements; Oracle, OCI and native AWS/Google SDK bundles enabled. Local in-process embeddings disabled, matching live.
Model and application state	Existing provider mappings, remote Ollama endpoints, reranker files, encryption keys, runtime settings and storage routes preserved.

Do not change Docker build flags, base digests, dependencies or model configuration during a release to work around a failed preflight. Keep required providers and any deliberately selected laptop model relay available. A powered-off laptop cannot serve its Ollama model; Gemini-mapped datasets use their existing cloud provider.

The main entry point is deploy.ps1. The application image helper, database installer and read-only guard remain implementation/support tools. Do not bypass the approval workflow by running an underlying publish helper directly.

5 Deploy using the AI skill

Give the AI your source location/ref, target JSON location and component selection. Use the repository skill at `.agents/skills/aegis-deploy/SKILL.md`. If the tool recognizes installed skills, invoke **$aegis-deploy**; otherwise ask it to read and follow that file.

Copy this request to the AI

Use $aegis-deploy and follow .agents/skills/aegis-deploy/SKILL.md in my Aegis checkout. Deploy the tested ags_sprint1 source using C:\aegis-target\target.json. Use its DB_Object, DB_ReleaseFile, DeployFrontend and DeployBackend selections. Preserve all existing OCI infrastructure, application configuration, model registrations and data. Do not provision infrastructure or change settings. Ask me for missing inputs; if unknown, request permission for bounded read-only discovery. Before publishing, show the complete verified release plan and wait for my explicit approval.

The AI must complete these stages

1. Resolve inputs locally. Check source, destination file and selected components. Ask about unknown values. Sharing a tenancy ID does not select a unique cluster or database, and the AI must not guess the target.

2. Obtain connection permission. Confirm the account/profile, tenancy, region and discovery scope before OCI, Kubernetes, Oracle or public-target checks. Existing consent for the same release scope can be reused. Discovery is read-only and does not permit publication.

3. Prepare a concrete release plan. Verify target identity and prerequisites; complete applicable source checks and local image builds. Show the exact commit, cluster/context/server, namespace, architecture, registry, URL, current and proposed images, DB owner/full service, script hashes, missing objects, any new schema/grants/quota, active jobs, preserved configuration fingerprints, evidence, unrun checks and expected restart window.

4. Obtain final approval. Wait for explicit approval of that exact plan before DDL, image pushes or workload updates. Credential access, read-only consent and successful preflight are not publication approval. Recheck changed source/target details and obtain amended approval if the plan changes.

5. Execute and report. Apply only approved missing DB objects and selected images, verify configuration and readiness, and return source/target, DB outcome, image digests, HTTPS URL, exact receipt and any limitations. Stop dependent work on failure; do not automatically retry or roll back.

Use a private credential mechanism or the operator terminal for secrets. Do not paste passwords or tokens into the AI request. An unknown application URL may be discovered from existing routing after permission; the skill must stop if routing or other initial setup is missing.

6 Deploy using JSON and PowerShell

Open an interactive PowerShell terminal at the Aegis repository root. This route runs directly without an AI agent. Complete target input preparation on pages 2-4. For an application release, ensure Docker is running and your existing destination registry login is valid.

If registry login is needed, run the command below using the destination registry host. Use Docker's interactive password prompt for the approved registry token; never add it to a command or JSON. This authenticates the local client and does not publish images.

```
$taskRegistry = 'YOUR_DESTINATION_REGISTRY_HOST'
docker login $taskRegistry
```

Optional read-only preflight

```
.\ags_deploy\deploy.ps1 `
  -Inputs C:\aegis-target\target.json -CheckOnly
```

Confirm the displayed target and type **CONNECT** only when read-only access is permitted. Supply private database passwords if prompted. CheckOnly performs remote/schema preflight without builds, DDL, image pushes or rollout. Review the printed result and resolve any prerequisite failure. It does not grant approval for a subsequent deployment.

Run the release

```
.\ags_deploy\deploy.ps1 -Inputs C:\aegis-target\target.json
```

1. Review the source and target; type **CONNECT** to authorize read-only discovery for this run. Missing non-secret inputs are prompted or discovered. Database passwords use private terminal prompts.

2. Read preflight results. If app images are selected, the command builds them locally using the exact committed source. ValidationReference records your source test/CI evidence; tests are not run by this command.

3. Review DEPLOYMENT PLAN: exact commit, target, component selection, image IDs/tags, database owner/full service, missing-object/new-schema actions, conditional readiness and preserved settings. Type **PUBLISH** only when you approve those exact changes. Declining stops publication.

4. The command rechecks source, target and artifacts, applies approved DB actions first, verifies the saved connection and readiness, then pushes and updates only selected images. Backend updates API and worker together; frontend follows when selected. Drift or failures stop dependent steps.

5. Keep the exact Application, Receipt and Record paths printed by this invocation. Successful app execution verifies HTTPS 200, selected images, readiness, schema and preserved configuration. Review result.json before declaring completion.

No backup reference is mandatory or prompted for. Do not use legacy DatabaseAction examples alongside DB_Object. The current Y/N template and this entry point are the supported handover workflow.

7 Verify and hand over the result

Retain the private run directory under ignored `ags_deploy/deployment-records/`. Use the exact paths printed by the run, rather than selecting a receipt by the newest filename. Keep destination details and records outside public Git history.

Record	What to confirm
Source and destination	Exact source commit, approved target/context/namespace, DB owner/full service and selected flags.
Database result	Created or skipped objects; new schema only if approved; final compatibility check. Database-only completion does not claim an application rollout.
Application result	Previous/new image digests, selected workload readiness, unchanged unselected images and protected configuration, existing HTTPS URL/status.
Functional acceptance	Check sign-in and read-only screens. A new upload, extraction or cited-chat test needs a prepared test workspace and explicit authorization for writes/inference.

If the command stops

Stop and inspect the recorded phase, completed DB objects and actual image versions. Oracle DDL can commit partially. A failed frontend rollout may follow a successful backend rollout. Do not blindly rerun, replay the fresh installer, run rollout undo, change settings, or claim a partial release succeeded. Recovery or restoration of previous compatible images requires its own approved action; no automatic rollback is provided.

Validation recorded on 16 October 2026

The checked source baseline below passed 1,047 backend tests (5 skipped), 167 frontend unit tests, frontend type checks and 69 offline deployment tests. ARM64 production images built successfully; frontend startup/assets returned HTTP 200; all 105 installed backend package versions matched live. Read-only Oracle comparison matched 13 tables, 165 columns, 49 foreign keys and 20 explicit indexes, with no missing listed objects.

Live health remained unchanged during that review. No live provider inference, upload/extraction/chat write workflow, new-tenancy installation or recovery drill was performed. These results support that source snapshot; a different target and changed code still need their own verification. A branch push is not deployment and is not a new CI run.

Verified application/tooling baseline: `3ae03a5f40fa24bd7c05f84892a9d73efddc677d`. The final handover adds documentation only. The original immutable tag aegis-freeze-2026-10-15 remains unchanged.

Maintained source and reference

Release entry point: ags_deploy/deploy.ps1
Input template: deployment-inputs.example.json
Object list: ags_database/release.json
AI workflow: aegis-deploy skill on ags_sprint1

Architecture and the approved Mural diagram are maintained separately in Aegis_Architecture_Handover.docx and its matching PDF. Pages 9-14 add the resource/model prerequisites, pilot provisioning example and first-install handoff. This document replaces the separate database, backend and Kubernetes deployment guides.

Resource provisioning reference

This section helps the platform/application owner prepare a **new** destination, separately from the code/object release on pages 1-8. Pilot values were recorded on 15-16 October 2026; verify availability, sizing and cost for the new target. Leave existing live resources unchanged.

OCI prerequisites and purpose

Resource or prerequisite	Purpose	Pilot example and requirement
Tenancy, compartment, region and IAM	Select the authorized target; permit resource management, cluster access, image push/pull and bucket operations.	Required. Mumbai ap-mumbai-1; approve the new compartment, identity, limits and budget before creation.
VCN, subnets, routing and security rules	Connect the API endpoint, workers and public entry; allow required database, OCI and model traffic.	Required. ags-aegis-vcn 10.42.0.0/16; API .0/24, workers .1/24, load balancer .2/24.
OKE cluster and managed node pool	Run frontend, FastAPI, agents, worker, HTTPS and CPU reranker.	Required for this hosting design. Basic ags-aegis-basic; 1 VM.Standard.A1.Flex ARM64 node, 2 OCPUs, 12 GB RAM; observed Kubernetes v1.36.4.
Boot and persistent block storage	Boot the worker; retain protected app state/model assets and certificate state across pod replacements.	Required. 50 GB boot; application-state and https-state claims each request 50 Gi, backed by 50 GB OCI volumes. oci-bv storage class.
Autonomous AI Database and wallet	Store app records, access, durable jobs and Oracle native vectors; authenticate the DB connection.	Required. ags_ora_db / agsora, Oracle 26ai, observed 1 ECPU and 20 GB; ADMIN, agsora_low, mTLS wallet. Not a new-target size default.
OCI Object Storage bucket	Store original documents, prompts, parsed content and versioned JSON/Parquet results.	Required. Standard bucket mk-aegis; use the new target's namespace, bucket and authorized credentials.
OCIR repositories and image access	Store immutable application images and let OKE pull them.	Required. Private ags-aegis-web and ags-aegis-backend. Pilot also retains ags-aegis-ssh for its model relay.
Public load balancer, DNS and HTTPS	Give users the application URL and terminate HTTPS through the web pod's Caddy sidecar.	Required for public access. Pilot flexible load balancer 10 Mbps min/max, ports 80/443; HTTPS state is persistent.
Kubernetes namespace, Secrets and ConfigMaps	Provide workload identity/access, DB wallet, encryption key, settings, image pulls and target routes.	Required. aegis-oci; exact protected-state and resource contracts are listed on page 13.

The pilot DB Free Tier flag does not make the whole environment free; the observed 1 ECPU allocation is not a new-instance sizing default. The Gemini path requires no OCI Generative AI service.

Model and operator prerequisites

The three inspected active datasets use Google Gemini. Arrange its provider account, endpoint, credentials and quota separately from OCI. Oracle native VECTOR stores embeddings; a model generates them. Confirm model availability and parameters for the new provider account.

Role	Purpose	Observed model and required setup
Embedding model	Create document/query vectors for semantic retrieval and indexing.	Required for knowledge indexing/search. Saved model gemini-embedding-2 via Google Gemini API; a valid embedding registration, credentials and dataset mapping are needed.
Chat model	Generate evidence-based Unified Chat answers and citations after retrieval/reranking.	Required for chat. Saved gemini-3.5-flash-lite; register the Chat role and map it to the dataset.
Extraction and OCR model	Return structured extracted fields; read scanned/image content when OCR is needed.	Required for extraction and relevant scans. Saved gemini-3.5-flash-lite in the Extraction role; dataset-aware OCR uses that mapped registration.
CPU reranker	Rank retrieved evidence before chat/extraction generation.	Required for the production workflow. cross-encoder/ms-marco-MiniLM-L6-v2; pinned ONNX and tokenizer assets at /data/models/reranker inside OCI API/worker state.
Laptop Ollama alternative	Serve local embedding/generation models when deliberately selected instead of the cloud path.	Optional. Retained Qwen3 chat/extraction and Nomic embeddings; Qwen vision is an alternative OCR setting. Requires matching weights, powered-on laptop, Docker/Ollama and working relay. Not required by Gemini-mapped datasets.

Release computer and protected runtime state

Prerequisite	Purpose and when needed
Git, PowerShell and Python	Read the committed source and run deploy.ps1. PowerShell 5.1+, Python 3.10+; 3.12 recommended. DB object work needs python-oracledb from requirements-oracle.txt.
Docker, kubectl and OCI authentication	Build target-platform Linux images, push/pull them, inspect and update OKE. Required for application releases; OCI CLI or the kubeconfig's approved authentication helper must work.
Workspace state, wallet and keys	Keep saved target DB/storage settings and encrypted secrets usable after restart. Use a new stable workspace key for a new installation; preserve that key on later releases.
Network and model access	Allow target access to DB/wallet service, Object Storage, OCIR and required model endpoint. Check provider quota and reranker integrity; full inference requires an authorized test.

The pinned reranker revision is 233902d25c440f23af6f7d6e94d2946bac0bee0a. The checked-in reranker module verifies model/tokenizer integrity; the release command does not download or replace these files. No dedicated GPU, Qdrant or separate vector database is part of the observed OCI/Gemini design.

A Provision networking and OKE

How our pilot was provisioned. The saved provisioning record confirms VCN, Internet Gateway, route table, three security lists/subnets, a Basic OKE cluster and the A1 node pool were created through the retained Python/OCI SDK helper. It recorded request IDs and retry tokens, checked Mumbai/home-region and capacity assumptions, and stopped rather than selecting a paid fallback. The maintained historical source is ags_deploy/ags_oke/ags_legacy_provisioning.py.

The helper is specific to that pilot and its private state; do not run its create operation unchanged in another tenancy. The steps below explain equivalent destination preparation in OCI Console. They are separate provisioning actions, requiring the platform owner's target, sizing and configuration approval.

1 Select the destination and permissions

Record the tenancy, compartment, region, authorized user/profile and limits. Have the OCI administrator grant the approved permissions for networking, OKE/Compute, volumes, registry and bucket access, plus Kubernetes access. Check capacity and estimated charges. Use new names/IDs for a new environment and record every created resource.

2 Create the VCN and subnets

In Networking, create a VCN with non-overlapping address ranges. The pilot uses 10.42.0.0/16 with API 10.42.0.0/24, worker 10.42.1.0/24 and load-balancer 10.42.2.0/24 subnets. Flannel pod/service ranges are 10.244.0.0/16 and 10.96.0.0/16. Its public subnets use an Internet Gateway and default route. For a new private-node design, arrange the appropriate NAT/service gateways and management route. [1]

Create the destination security lists or NSGs for OKE control-plane/worker communication, approved management access, app ports 80/443 and required outbound traffic. In the pilot, management 6443/22/32022 is restricted to the recorded operator /32; 32022 belongs only to its optional SSH model relay. Do not expose the Oracle database or Ollama model port as public application ports.

3 Create the cluster and node pool

In Kubernetes Clusters, choose Create cluster and Custom create, select the prepared network, and use Basic when the chosen features permit it. The pilot used Flannel, a public API endpoint, and one managed ARM64 VM.Standard.A1.Flex worker with 2 OCPUs, 12 GB RAM and a 50 GB boot disk. Select an available supported Kubernetes version and matching OKE node image for the destination; v1.36.4 is the observed pilot version, not an availability promise. [2]

4 Obtain access and confirm the result

Wait for the cluster and worker to become available/Ready. Use Access cluster > Local Access to generate a destination-specific kubeconfig with the authorized OCI identity. Private APIs require the approved network/bastion path. Record context, server and architecture, then inspect nodes and storage classes. Do not reuse the pilot's private kubeconfig helper/state for a new tenancy. [3]

[1] Oracle OKE networking | [2] Oracle Custom create | [3] Cluster access

B Provision database storage and registry

Our pilot example: Aegis connects to the existing Autonomous AI Database agsora and Standard Object Storage bucket mk-aegis. The OKE provisioner does not create either service. Their connection/configuration was verified; this handover does not claim a recorded original Console creation sequence for them.

1 Create the Autonomous AI Database

In Oracle AI Database > Autonomous AI Database, select the destination region/compartment and Create Autonomous AI Database. Enter new display/database names, select the approved workload, and choose Oracle AI Database 26ai with native VECTOR/BOOLEAN and compatible collation support. Set the permitted compute/storage, scaling, network access and private ADMIN password; review charges and create. Wait for AVAILABLE. The pilot observed 1 ECPU/20 GB with the Free Tier flag; a new target must use the options actually available to it. [4]

Choose the network path permitted by the destination policy and ensure both the operator and OKE can reach it. Under Database connection, download that database's wallet into a private directory. Record its TNS alias and full SERVICE_NAME; keep the wallet password private. Retain mTLS for a wallet-based installation. [5]

2 Create the application objects

Use pages 3-5 with the new wallet, owner, admin identity and service. For a database-only run, set DB_Object Y and both app flags N, and review the missing schema/grants/quota and ordered object plan before PUBLISH. This creates only supported missing objects in the already provisioned database. An incompatible database capability or existing schema must be resolved before proceeding; the release tool does not change database-wide settings.

3 Create the document bucket

In Object Storage, choose the destination compartment and Create bucket. Use a target-specific name, Standard tier and the approved access/versioning choices. Record region, namespace and bucket, then grant the application identity the needed object operations. The inspected pilot bucket had ObjectRead public access and versioning Disabled; those are observations, not required application settings. Choose the new target's approved policy and do not change live to copy this example. [6]

4 Prepare image repositories and credentials

In Container Registry, create private ags-aegis-web and ags-aegis-backend repositories in the approved destination compartment. Retain a separate relay image only if that architecture is selected. Record the registry domain and tenancy namespace; the pilot domain is bom.ocir.io. Configure authorized operator push access and Kubernetes pull access. Docker login on the laptop does not create the namespace's image-pull Secret. [7]

[4] Create Autonomous AI Database | [5] Download wallet
[6] Create Object Storage bucket | [7] Private image pulls

C Prepare the initial application installation

A new cluster needs a reviewed destination-specific first-install package. The code-release command only updates existing workloads. The pilot renderer, ags_deploy/ags_oke/ags_render.py, generates manifests from private state; it does not apply them and its init container expects already prepared OCI configuration and reranker files. It also assumes the pilot hostname/relay design. Review/adapt that package before first installation; do not run it blindly for a new target.

Initial installation item	What the platform/application owner prepares
Namespace and workloads	aegis-oci; aegis-web with web and https; aegis-api-worker with api and worker using the same compatible backend image. The pilot also has a model-relay sidecar. Use target-platform image digests and the reviewed single-replica Recreate contract.
Persistent volumes	application-state and https-state claims with oci-bv; pilot 50 Gi each, ReadWriteOnce. API/worker share /data. Arrange UID/GID 10001 access and compatible scheduling. CSI provisions backing OCI block volumes when claims bind. [8]
Secrets and mounts	ocir-pull; aegis-app; oci-credentials; database-wallet mounted read-only at /run/aegis-db/wallet. ssh-relay only for an approved relay design. Supply secret payloads privately, not in Git or command arguments.
Protected saved state	Destination configuration/database.json with active_provider=oci, active_connection and a valid encrypted secret_ref; configuration/settings.json with storage_provider=oci and target bucket route. Use independent destination credentials and a new stable workspace encryption key; preserve its relationship to encrypted state.
Runtime and model files	aegis-config, aegis-runtime-config and OCI runtime profile; CPU reranker assets in /data/models/reranker. Do not use local Compose/container_prepare as an OCI initializer: that separate path expects other local services.
Web routing and certificates	Internal api ClusterIP on 8000; web on 3000; Caddy reverse proxy and persistent certificate state; aegis-public LoadBalancer exposing 80/443. The reviewed pilot manifest requests a flexible 10/10 Mbps load balancer.

How the URL appears: the initial LoadBalancer Service obtains its public address from OCI. Record it, configure the approved DNS hostname and Caddy HTTPS host, then verify certificates and reachability. The pilot uses an aegis.<public-IP>.sslip.io hostname. This first-install DNS/TLS configuration is outside deploy.ps1; later code releases discover and preserve the existing URL.

The platform owner installs only the reviewed package for the approved new destination, then checks bound storage, correct mounts, images, configuration and readiness. No general unattended empty-tenancy or protected-state bootstrap installer is provided by the current release tools. If that package/state is missing, stop and prepare it before the release command.

[8] Oracle CSI block-volume claims

D Prepare models and complete the handoff

1 Install the pinned CPU reranker assets once

The deployed backend runs ONNX reranking on its CPU and expects onnx/model.onnx and tokenizer.json under /data/models/reranker. The maintained setup helper downloads the fixed model revision and verifies hashes. On an authorized preparation computer, install the shared Python requirements and prepare assets into a staging directory:

```
.\.venv\Scripts\python.exe -m pip install `
  -r ags_backend\requirements.txt
.\.venv\Scripts\python.exe `
  .\ags_local_use\ags_scripts\setup-reranker.py `
  --directory C:\aegis-target\reranker
```

The platform owner transfers those verified files into the **new destination's** application-state volume at models/reranker, with the required permissions, before startup. The staging command alone does not install files in OCI. Model downloads occur only during this initial setup; code releases preserve the installed assets.

2 Register and map the chosen cloud models

Arrange a permitted Google Gemini endpoint, private API credential and sufficient quota for the new destination. In the initialized Aegis workspace, register and test the Embedding, Chat and Extraction roles, then map the dataset to them. The pilot's saved examples are gemini-embedding-2 and gemini-3.5-flash-lite; confirm that the actual provider account supports the chosen identifiers and parameters. OCR for the inspected Gemini datasets uses the mapped Extraction registration, not the retained local OCR default.

Use a compatible embedding configuration and dimensions for the destination's native vectors; changing an established embedding mapping can require reindexing and is not a code-release action. Complete authorized workspace administrator/bootstrap setup if the new workspace is empty. New model registration, dataset mapping, key setup and any validation inference belong to this initial installation scope.

3 Keep local models optional

Only if the selected design uses laptop Ollama, prepare its required model weights and approved SSH relay and verify cloud-to-laptop reachability. The laptop, Docker/Ollama and relay must remain online for those requests. No laptop model is required by the observed Gemini-mapped datasets. Local container health is not proof of a working relay or model inference.

4 Hand the prepared destination to the release operator

Record the resource inventory, context/server, namespace, registry, architecture, DB owner/full service/wallet alias, bucket route, HTTPS URL and model availability. Verify the app's protected saved state, bound volumes, reranker integrity and readiness. A harmless upload/index/extract/cited-chat acceptance test requires a test dataset and authorized writes/provider calls; record what actually passed.

Fill the non-secret JSON on page 3 using this target. Run the page 7 read-only preflight; after it passes, use the AI or terminal route with the exact component flags and final release approval. Provisioning is now complete; the subsequent release changes only the reviewed missing DB objects/schema and selected application images.

This provisioning reference was checked against retained pilot source/creation records and linked Oracle documentation on 16 October 2026. No resources, configuration, database objects, model registrations or images were changed to prepare it; no new-tenancy installation was rehearsed.